

Stoquify Executable Copilot Runtime Backbone Requirements

Architecture and implementation readiness report

16 July 2026 | Controlled blueprint

The installed agents and skills are the specification layer. This report defines the secure, durable and observable runtime required to turn them into a production Copilot.

Executive summary

Stoquify does not need a second application architecture or a disconnected AI platform. It needs a controlled runtime layer that places the installed Copilot agents and skills inside Stoquify's existing security, entitlement, workflow, evidence, audit, and operational-control boundaries.

The target is a durable, tenant-safe, evidence-grounded and policy-controlled Copilot. It must support read-only assistance first, then progressively introduce human-approved actions only after evaluation, shadow operation and canary gates pass. PostgreSQL should remain the authoritative system of record. Redis should be limited to coordination concerns such as queues, leases, caching and streaming; it must never become the source of truth for runs, approvals or evidence.

The work is substantial but feasible. The definitions, contracts and evaluation catalogue already provide a strong design foundation. The next stage is an implementation programme covering the control plane, registry, runtime state machine, secure tool gateway, model gateway, durable storage, APIs, user experience, observability, evaluation and phased release.

1. Foundations Stoquify should reuse

The runtime should extend the controls already present in Stoquify rather than duplicate them.

Runtime requirement	Existing Stoquify foundation	Required extension
Tenant and role authorization	<code>lib/security/server-authz.ts</code>	Produce a signed or immutable server-derived Copilot context envelope for every run and step.
Module entitlement	<code>services/modules/module-entitlement.service.ts</code>	Apply entitlements to agent visibility, tool eligibility and data access; move from observation to proven enforcement through a controlled ratchet.
Sensitive actions	<code>services/controls/sensitive-action.service.ts</code>	Map every Copilot tool to an explicit action policy, risk class and approval requirement.
Step-up authentication	<code>services/security/step-up-auth.service.ts</code>	Bind fresh authentication to the exact plan, evidence, parameters and approval being authorized.
Business events and idempotency	<code>services/events/business-event.service.ts</code>	Add versioned Copilot run, step, tool, approval, outcome and incident events with replay-safe semantics.
Evidence and redaction	<code>services/evidence/*</code>	Add Copilot-specific provenance adapters, fingerprints, trust grades, freshness checks and disclosure policies.

Runtime requirement	Existing Stoquify foundation	Required extension
Assurance and incident controls	services/assurance/*	Detect policy denials, agent drift, excessive cost, evidence failures, unsafe actions and runtime degradation.
Audit	AuditLog	Store immutable run, approval, tool-invocation, policy-decision and outcome traces.
Notifications	Existing notification services	Deliver approval requests, blockers, completion notices, suspensions and incidents through approved channels.
Runtime coordination	PostgreSQL and Redis	Keep durable truth in PostgreSQL; use Redis only for non-authoritative coordination.

2. Target runtime backbone

The target backbone is a policy-governed execution spine with clear boundaries:

1. A Copilot API receives an authenticated request.
2. The context service derives tenant, identity, permissions, entitlements and operating scope from trusted server-side sources.
3. The policy engine decides which agents, skills, models, data and tools are eligible.
4. The registry resolves compatible, enabled and pinned capability versions.
5. The dispatcher creates a durable run and advances it through a deterministic state machine.
6. Evidence services retrieve, filter, redact, grade and fingerprint supporting information.
7. The model gateway selects an approved model within privacy, regional, budget and latency constraints.
8. Any external or internal operation passes through the deterministic tool gateway.
9. High-risk operations pause for properly separated human approval and fresh authorization.
10. Every decision, transition and outcome is recorded, measured and available for assurance review.

3. Durable runtime data model

The following records should be persisted in PostgreSQL with tenant isolation, timestamps, version identifiers and immutable audit references:

- **CopilotCapabilityVersion**: agent or skill identity, semantic version, checksum, dependencies, compatibility range and lifecycle status.
- **CopilotRun**: tenant, actor, purpose, capability versions, policy snapshot, risk class, state, budget and correlation identifiers.
- **CopilotRunStep**: ordered step, input and output references, assigned capability, model/tool use, state, attempts and timing.
- **CopilotCheckpoint**: compare-and-set version, resumable state, prior checkpoint hash and recovery metadata.
- **CopilotEvidenceReference**: source, record identifier, provenance, freshness, trust grade, classification, redaction status and fingerprint.
- **CopilotApprovalRequest**: maker, eligible approvers, separation-of-duty rule, exact action digest, expiry, revocation and consumption state.
- **CopilotToolInvocation**: registered tool version, validated parameters, idempotency key, authorization result, execution state and outcome query data.
- **CopilotOutcome**: delivered recommendation or action result, confidence, evidence links, limitations, feedback and downstream effect.
- **CopilotEvaluationResult**: case version, capability versions, environment, scores, failures, reviewer and release-gate decision.

- [CopilotTenantPolicy](#): enabled capabilities, autonomy ceilings, data rules, budgets, model restrictions and feature flags.
- [CopilotModelPolicy](#): approved providers/models, regions, retention rules, sensitivity limits, quotas, fallbacks and suspension state.
- [CopilotIncident](#): severity, affected tenants/runs, trigger, containment, evidence, resolution and follow-up actions.

Redis may hold queue entries, leases, short-lived caches and streaming state. All such entries must be reconstructable from PostgreSQL and business events.

4. Capability registry and loader

Create [services/copilot/registry](#) as the authoritative loader for the installed agent and skill packages. It should:

- Load the current capability catalogue and validate schemas, checksums, dependencies and compatibility before activation.
- Pin every run to exact agent, skill, prompt, tool-contract and policy versions.
- Reject missing, incompatible, disabled, quarantined, deprecated or suspended capabilities.
- Expose lifecycle states such as draft, validated, shadow, canary, active, suspended and retired.
- Support tenant-specific enablement without allowing tenants to exceed platform safety ceilings.
- Permit canary activation and immediate rollback to a previously approved version.
- Record who approved each promotion and the evaluation evidence supporting it.

Registry loading should fail closed. A partially valid catalogue must not silently expose an unsafe subset.

5. Trusted context and policy control plane

Create [services/copilot/context](#) and [services/copilot/policy](#). The trusted context must be derived on the server and include:

- Tenant, organization and legal-entity scope.
- Actor identity, authenticated session and assurance level.
- Roles, permissions, delegated authority and segregation-of-duty constraints.
- Location, branch and operational scope.
- Enabled modules, packages and commercial entitlements.
- Country pack, accounting period and applicable compliance context.
- Locale, language and time zone.
- Feature flags, pilot cohort and capability restrictions.
- Data classifications, privacy limits and residency requirements.
- Risk and autonomy ceilings for the requested operation.

Client-supplied tenant IDs, roles, entitlement claims, risk levels or approval status must never be trusted. Context should be snapshotted for the run, revalidated before sensitive steps and included in the audit trail without exposing secrets.

6. Durable dispatcher and state machine

Create [services/copilot/runtime](#) to orchestrate capabilities without allowing uncontrolled agent-to-agent recursion. The dispatcher should provide:

- Explicit composition plans and dependency ordering.
- A normative state-transition table with fail-closed invalid transitions.
- Optimistic concurrency or compare-and-set checkpoints.
- Duplicate, loop and recursion prevention.

- Idempotent retry and safe resume after worker, network or model failure.
- Dead-letter and manual-recovery paths.
- Cancellation, suspension and tenant/global kill-switch handling.
- Dependency failure propagation so downstream work cannot continue on invalid inputs.
- Run, step, tool and approval timeouts with unambiguous terminal states.

The dispatcher should be deterministic around control flow even when model output is probabilistic. Models may propose; the runtime validates, constrains and decides what can proceed.

7. Deterministic tool gateway

Create [services/copilot/tools](#) as a closed, server-owned registry. Agents must not call Prisma directly, arbitrary service methods, provider SDKs, unregistered URLs or user-defined executables.

Every tool contract should declare:

- Stable tool ID, owner and version.
- Exact input and output schemas.
- Allowed service boundary and operation.
- Read, draft, controlled-write or prohibited classification.
- Required permissions, module entitlements and data scope.
- Timeout, retry and concurrency rules.
- Idempotency-key construction and deduplication window.
- Evidence and audit requirements.
- Approval and step-up requirements.
- Compensation or rollback procedure where possible.
- Post-timeout status query for ambiguous outcomes.
- Error taxonomy and safe user-facing explanation.

Start with read-only tools. Draft-generating tools may follow. Controlled writes should be introduced individually only after security review, executable evaluation, shadow evidence and canary approval.

8. Evidence, retrieval and content isolation

Create [services/copilot/evidence](#) to ensure that every material statement or proposed action is grounded in authorized business information. The service should:

- Retrieve only the minimum necessary fields and records.
- Apply tenant, role, entitlement, purpose and field-level filters before model exposure.
- Redact or transform sensitive fields before sending content to a model.
- Preserve source provenance, timestamps, freshness, trust grade, data classification and fingerprints.
- Detect stale, incomplete or mutually contradictory evidence.
- Treat retrieved instructions as untrusted data and quarantine prompt-injection content.
- Separate authoritative records from commentary, attachments and user-provided narrative.
- Provide citations that the UI can resolve without granting broader access.

A vector database is not required for the first production slice. Begin with authoritative structured sources and deterministic queries. Introduce semantic retrieval only when a measured use case justifies the additional security, freshness and operational burden.

9. Model gateway and cost controls

Create [services/copilot/models](#) as the only path to model providers. It should enforce:

- An approved provider and model registry with pinned versions or deployment identifiers.
- Regional routing and data-residency restrictions.
- Sensitivity-based model eligibility and redaction requirements.
- Per-request, per-run, per-user and per-tenant token and monetary budgets.
- Latency limits, concurrency quotas and rate limits.
- Structured-output validation and bounded repair attempts.
- Fallback models and provider circuit breakers.
- Zero-retention or equivalent contractual settings where required.
- Cost, latency, refusal, error and quality telemetry.
- Model, provider, tenant and global suspension controls.

The architecture should remain provider-neutral, but production should begin with one explicitly approved provider/deployment. Multi-provider failover should be added only when governance and evaluation coverage are mature.

10. Approval and human-control service

High-risk actions require a durable approval service, not a conversational confirmation. It must support:

- Maker-checker separation and configurable eligible-approver rules.
- Delegation controls and conflict-of-interest checks.
- Fresh authentication for sensitive approvals.
- Expiry, revocation and one-time consumption.
- Cryptographic or canonical binding to the exact action, parameters, evidence, capability versions and policy snapshot.
- Revalidation of permissions, entitlements, evidence freshness and target state immediately before execution.
- Immutable decision records including rejection reasons.
- Human-only operations that cannot be delegated to an agent under any autonomy setting.

Approval replay, self-approval, parameter substitution and execution after material evidence change must be rejected.

11. Copilot API and user experience

The first API surface should include:

- [POST /api/copilot/runs](#) to create an authorized run.
- [GET /api/copilot/runs/{id}](#) to retrieve permitted status and results.
- A server-sent-events endpoint for bounded progress streaming.
- Approval list, detail and decision endpoints.
- Feedback and outcome-correction endpoints.
- Administrative endpoints for registry status, evaluations, cost, feature flags and emergency controls.

The user experience should provide a command drawer, focused Copilot workspace, evidence and confidence views, approval inbox, run history, audit visibility and an administrator control centre. Every view needs complete loading, empty, denied, disabled, stale, partial, blocked, awaiting-approval, failed, cancelled and suspended states. English and French support should be built into contracts and evaluations rather than added after launch.

12. Observability, feature flags and kill switches

Operational telemetry should cover:

- Run and step success, failure, cancellation and resume rates.
- Policy denials and attempted boundary violations.
- Evidence coverage, freshness and contradiction rates.
- Model cost, tokens, latency, retries and fallback use.
- Tool latency, duplicate suppression, ambiguous outcomes and compensation.
- Approval latency, expiry, rejection and separation-of-duty failures.
- User feedback, accepted recommendations and corrected outcomes.
- Capability drift and evaluation regression.

Controls should include a global kill switch, tenant and capability flags, model and tool suspension, cohort canaries, hard budgets, automatic shutdown thresholds and tested rollback procedures. Each severe alert needs an owner and runbook.

13. The 999-case evaluation programme

The existing catalogue must be converted from specification material into executable, versioned fixtures. Coverage should include:

- Nominal workflows for each agent and skill.
- Malformed inputs and schema boundary conditions.
- Cross-tenant access, RBAC and disabled-module attacks.
- Stale, missing and contradictory evidence.
- Prompt injection and malicious retrieved content.
- Dependency failure and incompatible capability versions.
- Approval replay, self-approval, expiry and parameter substitution.
- Duplicate events, concurrent steps, retries and ambiguous tool timeouts.
- English and French interactions.
- Country-pack and OHADA-context variations.
- Payroll and workforce privacy scenarios.
- Offline replay and delayed event delivery.
- Model, tool and capability suspension and rollback.

Evaluation results must record exact versions and be release-gating artifacts. Passing definition validation does not mean these runtime cases have passed; they can be executed only after the corresponding runtime components and test harness exist.

14. Recommended implementation sequence

1. Freeze contracts and vocabulary. Confirm lifecycle states, state transitions, event names, schemas and versioning policy.
2. Build the trusted control plane. Implement server-derived context, policy decisions, entitlements and immutable decision logging.
3. Add persistence, registry and dispatcher. Establish the durable run state machine and capability loading.
4. Integrate evidence controls. Add minimum-data retrieval, redaction, provenance, freshness and content isolation.

5. Introduce a read-only model gateway. Connect one approved provider and enable only the Command Agent and orchestration path initially.
6. Deliver the API and core user interface. Provide run creation, streaming, evidence, history, feedback and administration.
7. Make evaluations executable. Automate the 999 cases and connect results to CI and release gates.
8. Run shadow mode. Compare Copilot output to existing operations without influencing production decisions.
9. Launch a narrow read-only pilot. Start with Command, Cash Reconciliation and Inventory insights for approved tenants and roles.
10. Add controlled action drafts. Introduce one action at a time after explicit security, assurance, shadow and canary gates.

15. Team and governance requirements

The programme requires clear accountability across:

- A principal platform/backend engineer for the runtime and service boundaries.
- A security and authorization engineer for tenant isolation, RBAC, approvals and threat modelling.
- An AI runtime/evaluation engineer for model integration, structured outputs and executable cases.
- A frontend/product engineer for the Copilot workspace and operational states.
- QA and SRE ownership for concurrency, resilience, telemetry, incidents and rollout.
- Accounting, OHADA, compliance and payroll reviewers for domain correctness and human-only boundaries.

Before implementation, leadership should decide:

- The first approved model provider, deployment region and retention terms.
- Pilot tenants, users, roles and modules.
- Per-tenant and platform cost ceilings.
- Queue and worker deployment model.
- Primary Copilot entry point in the application.
- Country-pack content owners and approval process.
- Incident ownership, support model and on-call expectations.

16. Recommended defaults

- PostgreSQL is the durable source of truth.
- Redis is coordination infrastructure only.
- The model interface is provider-neutral; production starts with one approved provider.
- The first release is read-only and evidence-grounded.
- All identity, tenant, role, entitlement and risk context is server-derived.
- Pilot access is explicit, opt-in and feature-flagged.
- Every run is version-pinned, budgeted, observable and auditable.
- Every controlled write is individually registered, idempotent and approval-bound.
- Any missing policy, context, evidence or compatibility decision fails closed.

Conclusion

The Copilot definitions and skills form a credible specification layer, but they become a product only when placed inside this executable backbone. The immediate objective should be a thin, read-only production slice that proves tenant isolation, evidence grounding, deterministic control flow, model governance and operational recovery. Once those foundations pass executable evaluations and shadow testing, Stoquify can safely expand capabilities and introduce carefully controlled actions.

The appropriate next deliverable is an implementation design package containing the database schema, service interfaces, event contracts, API contracts, deployment topology, threat model, evaluation harness and first-pilot acceptance criteria.